DATA STRUCTURES H / FUDAN UNIVERSITY

HNSW Approximate Nearest Neighbor Search

Construction Ordering and Query Early Stopping

AUTHOR

Zhang Chi

COURSE

Data Structures H (CS20009h.01)

TERM

Fall 2025 / Academic Year 2025-2026

REPORT COMPLETED

January 11, 2026

PUBLIC PROJECT ARCHIVE / OCTOBER 2026

The original Chinese report body follows this cover unchanged.
The cover was replaced to omit the student identifier.
All measurements describe the historical coursework submission.
See the repository documentation for measurement context.

1 方法介绍基线方法与问题分析

1.1 基线分析与方法介绍

（Hierarchical Navigable Small World（HNSW）是一类典型的基于图的近似最近邻（ANN）方法。其核心思想是构建分层图结构：高层为稀疏导航图、低层为稠密近邻图。构建阶段采用增量插入：每插入一个新点，先在当前图上进行多层贪心下降，再在第 0 层进行近邻候选搜索并选择若干邻居连边；查询阶段则从最高层入口点开始贪心下降到第 0 层，再以 best-first 方式扩展候选集，最终输出 top-K 结果。

HNSW 的强项在于“可导航性”：通过少量扩展即可快速接近查询点邻域。然而，在百万级数据规模与高维空间上，HNSW 的性能往往受到构建过程的随机性与底层扩展代价的双重影响。

1.2 HNSW 的主要不足（工程视角）

尽管 HNSW 在多种数据集上表现优秀，但在本任务设定下存在以下实际瓶颈：

1. **对插入顺序敏感，导致图质量波动**

   HNSW 的增量插入意味着“早期插入点”更容易成为图中的枢纽节点，后续节点的连边往往建立在已有结构上。若插入顺序与数据分布存在偏置（例如聚类结构逐步显现、难区域后插入），则容易出现局部区域连通性弱、近邻边被启发式剪枝误删等现象，最终带来 recall 的不稳定与查询阶段的冗余扩展，由于 glove 数据集本身所具有的数据分布特性，这一点在 glove 上的影响尤为明显。

2. **构建质量与构建代价之间存在显著 trade-off**

   构建参数（例如 efConstruction、M、M0）越大，插入时能在更充分的候选中选边，图质量更高，但构建时间急剧上升；反之构建更快但图更“噪”，查询需要更多扩展弥补，导致距离计算次数上升。

3. **查询阶段存在尾部冗余扩展（距离计算为主导开销）**

   在第 0 层 best-first 扩展过程中，即便 top-K 已经相对稳定，候选队列仍可能持续弹出并扩展大量低收益节点。由于距离计算通常为查询的主要成本，这类冗余扩展会显著增加平均距离比较次数与整体延迟。即，最近的几个点可以很快被找到，但是最后的几个点往往需要多次震荡后才最终能被发现

1.3 方法概览

为同时改善“图质量稳定性”与“查询阶段效率”，本文在 HNSW 框架上引入以下改进模块：

- **难度感知的构建预处理：LID 引导的插入顺序重排（two-pass build）**
- **第 0 层后处理：反向边注入（reverse-edge injection）与度约束剪枝（L0 prune）**
- **查询阶段策略：基于 γ的早停与候选队列约束（gamma early-stop + bounded candidate）**
- **实现层优化：SQ16 量化 +AVX2 距离、邻接表扁平化、CPU 预取、并行构建与线程本地 visited**

下文分别给出各模块的设计与理论直觉。

1.3.1 构建预处理：LID 引导的插入顺序重排

1.3.1.1 设计动机

不同区域的数据“局部难度”不同：在局部维度更高、密度变化更复杂的区域，少量近邻边更难保证可导航性，搜索更容易陷入局部最优。我们采用局部内禀维度（Local Intrinsic Dimensionality, LID）作为“难度指标”：LID 越大，表示该点邻域的距离分布更“陡峭/复杂”，在图上更需要高质量的近邻连接作为骨架。若这些“高难度点”在标准 HNSW 中较晚插入，则其连边只能在既有结构上局部补充，难以充当早期骨架的一部分，从而降低整张图的均衡性与稳定性。

1.3.1.2 两遍建图流程（Two-pass Build）

我们采用两遍建图以估计 LID 并重排插入顺序：

Pass-1（粗建图）：以较小的构建搜索宽度 efC_{lid}（代码中为 EF_LID_BUILD_）快速建一遍图，仅用于获得每个点在第 0 层的一批近邻候选。

LID 估计：对每个点 x，取其第 0 层邻接表中前 K_{lid}个邻居（代码中为 K_LID_），计算距离集合 $\{r_1, \dots, r_k\}$ 并排序 $r_1 \le \cdots \le r_k$。使用常见的 MLE 形式估计：

$$\widehat{\mathrm{LID}}(x) = -\frac{k-1}{\sum_{i=1}^{k-1} \log \frac{r_i}{r_k}}$$

其中 r_k为第 k近邻距离。该估计在 r_i接近 0 时数值不稳定，因此实现中引入 ϵ进行下界截断以保证数值安全。

Pass-2（正式建图）：按 $\widehat{\mathrm{LID}}(x)$降序对全体点排序，优先插入高 LID 点；随后恢复原构建参数（efConstruction、M、M0）重建最终图。

1.3.1.3 为什么有效（直觉与分析）

该策略可理解为“先搭骨架再填充”：高 LID 点更像图的难点区域，若先插入，其连边在早期图更稀疏的阶段更容易被保留为关键桥边；而低 LID 点后插入时可在已有骨架上补充局部连通，从而提升整体可导航性与稳定性。

从查询角度看，更均衡的骨架意味着在相同的 efSearch 下，best-first 更快进入正确邻域，减少无收益扩展。

1.3.2 第 0 层后处理：反向边注入与剪枝

1.3.2.1 反向边注入（Reverse-edge Injection）

构建过程中由于启发式剪枝与并行插入，邻接关系可能出现非对称：存在 $u \to v$但 $v \not\to u$。这会降低局部连通性，尤其在低层图上容易造成“局部断裂”，导致搜索过程中候选扩展震荡、回溯增多。

因此，我们在第 0 层进行一次后处理：遍历边 $u \to v$，若 v的邻接表中不存在 u，则将 u注入到 v的邻接表中，从而提高互惠性与可达性。

1.3.2.2 度约束剪枝（L0 Prune）

反向注入会增加部分节点度数。为保证内存与计算边界，我们在每次注入或最终阶段对度数超过 M_0 的节点执行剪枝：

先以距离对邻居排序并形成候选集，再采用多样性启发式（你代码中的 heuristic：若候选点与已选邻居过近、可能冗余，则丢弃）保留至多 M_0个邻居。该剪枝倾向于保留“既近又分散”的邻居集合，提高可导航性并避免局部团簇过密。

1.3.3 查询策略：γ 早停与候选队列约束

1.3.3.1 基本流程

在第 0 层查询中，我们维护两个堆：

- 候选堆 C：按距离从小到大弹出（min-heap），表示待扩展节点；
- 结果堆 B_K：保存当前最优的 K个候选（max-heap），其堆顶距离记为 d_k（即当前 top-K 中最差的距离）。

1.3.3.2γ早停准则

定义动态阈值：

$$\mathrm{bound} = (1+\gamma)\, d_k$$

当 C堆顶的距离 $d_{\min}$满足

$$d_{\min} > \mathrm{bound}$$

则停止扩展。这一准则的核心含义是：当当前 top-K 的“最差距离”已足够小，且候选堆中最有希望的节点

都已显著差于 $(1+\gamma)d_k$时，继续扩展的收益很低，更多是尾部冗余访问。

1.3.3.3 候选队列约束（Bounded Candidate Insertion）

为进一步减少无收益扩展，我们仅将满足 $d \le$ bound的新邻居入队 C。这等价于将搜索限制在一个自适应的“可收益区域”内，使候选扩展集中在更可能改善 top-K 的节点集合中。

1.3.3.4 为什么有效（分析）

在可导航图上，best-first 很快能够找到较优近邻，使 d_k快速收敛；此时 bound会收紧，γ 提供安全余量以平衡 recall 与速度。若图结构更均衡（由预处理与后处理带来），则 d_k更稳定、更早收敛，因此早停更“敢停且停得对”，从而显著降低距离计算次数，且不损害（甚至提升） recall。

1.3.4.实现优化

1.3.4.1SQ16 量化与 AVX2 距离

为降低内存带宽并提升距离计算吞吐，我们对原始 float 向量进行线性量化为 int16（SQ16）。距离计算采用 AVX2 指令对 int16 差值进行平方和累加（利用 _mm256_madd_epi16），在保持排序一致性的前提下显著加速大规模距离评估。

1.3.4.2 第 0 层邻接表扁平化与 CPU 预取

第 0 层邻接表采用扁平化布局（定长 stride），减少指针跳转与分散内存访问；在遍历邻居时使用 _mm_prefetch 对下一邻居的向量数据进行预取，以降低 cache miss 开销，提高查询阶段稳定性。

1.3.4.3 并行构建与 visited 优化

构建阶段采用多线程插入，节点级锁保护邻接表并发更新；搜索过程中使用线程本地 visited 版本号（token）避免每次 memset 清零 visited 数组，从而降低同步与内存写开销。

2 实验对比

2.1 lid 重排

在保持其余参数不变的情况下，我对比了 Baseline HNSW 与 引入 LID 引导插入顺序重排后的版本（LID-added）。实验结果表明，LID 重排对构建与查询性能产生了可控且具有积极意义的影响。

在构建阶段，LID-added 版本的构建时间由 424.1 s 增加至 458.4 s，增幅约为 8.1%。该额外开销主要来自两遍建图流程：第一遍用于估计各节点的局部内禀维度（LID），第二遍按照 LID 降序进行正式插入。这一代价属于一次性的离线成本。

在查询阶段，测试下来平均查询时间由 0.7588 ms/query 略增至 0.7709 ms/query，根据下文所述的平均距离计算次数的减少，可以认为这是测量时的抖动，而中位数查询延迟几乎保持不变（0.4042 ms → 0.4056 ms），说明主流查询路径的访问深度并未显著增加。与此同时，Recall@10 从 0.993 提升至 0.995，表明在高维、复杂区域中，优先插入高 LID 点有助于提升图结构对近邻关系的覆盖能力。

更值得注意的是，平均距离计算次数由 13202.77 降低至 12991.17，下降约 1.6%。由于距离计算是查询阶段的主要开销，该结果说明 LID 重排后得到的图在相同搜索参数下具有更好的可导航性，能够更快进入目标邻域并减少无收益扩展。总体而言，LID 引导的插入顺序重排在付出有限构建开销的前提下，提高了图结构的稳定性与搜索效率，为后续查询优化策略提供了更有利的结构基础。

组	buildtime	avg_ms	med_ms	recall	avgcmp
baseline	424.1	0.758759	0.404159	0.993	13202.77
lid_added	458.369	0.770949	0.405591	0.995	12991.17

2.2 反向边注入与剪枝

在该组参数设置下，我们对比了 Baseline HNSW 与 取消构图后处理（reverse-edge injection + L0 prune） 的版本。实验结果显示，关闭后处理后，构建时间由 424.1 s 显著降低至 275.8 s，平均查询时

间基本保持一致（0.7588 ms → 0.7582 ms），同时平均距离计算次数由 13202.8 降至 12970.3，Recall@10 也从 0.993 略微提升至 0.994。该现象表明，在该特定参数组合下，原始构图已经具备较好的局部连通性，后处理步骤可能引入了额外的边调整与剪枝开销，从而在统计意义上未体现出优势。

然而，需要指出的是，这一结论并不具有普适性。在其他参数配置下（如 MO=54 efC=500 gamma=0.21），构图后处理能够有效提升低层图的互惠性与均衡性，从而带来更稳定的 recall 提升，尤其是在复杂或稀疏区域中表现更为明显。综合多组实验结果可以发现，构图后处理在不同参数区间对性能的影响存在差异，但其在提升图结构鲁棒性与 recall 稳定性方面具有正向作用。因此，尽管在本组实验中关闭后处理表现略优，本文最终版本仍保留构图后处理步骤，以获得在更广泛参数范围内更稳健、可复现的整体性能表现。

组	buildtime	avg_ms	med_ms	recall	avgcmp
baseline	424.1	0.758759	0.404159	0.993	13202.77
no_post_process	275.763	0.758213	0.407492	0.994	12970.31

2.3gamma 早停与候选队列约束

在该组实验中，我们对比了 Baseline HNSW、关闭 gamma 早停（no_gamma_early_stop） 以及 保留 gamma 早停但取消候选队列约束（cand_always_enqueue） 三种配置，以分析查询阶段早停策略及候选控制机制的作用。

当完全关闭 gamma 早停时，平均查询时间显著上升至 1.2415 ms，增长超过 60%，平均距离计算次数也大幅增加至 20611.7。这表明在缺少早停准则的情况下，第 0 层 best-first 搜索会持续扩展大量低收益候选节点，导致尾部冗余扩展显著增加，距离计算开销成为主要瓶颈。

在保留 gamma 早停但取消候选队列约束的情况下（cand_always_enqueue），平均查询时间下降至 1.0785 ms，较完全关闭早停有所改善，但仍明显高于 Baseline；同时平均距离计算次数与 Baseline 基本持平。这说明 gamma 早停本身能够在一定程度上抑制无收益扩展，但若缺乏对候选入队的约束，候选堆中仍会累积大量边界节点，导致搜索过程震荡、延迟上升。

综合来看，gamma 早停与候选队列约束在查询阶段具有明显的互补关系：仅使用早停或仅限制候选均难以达到最优效果，二者结合才能在保持 recall 基本稳定的前提下，有效降低查询延迟并控制距离计算次数。

组	buildtime	avg_ms	med_ms	recall	avgcmp
baseline	424.1	0.758759	0.404159	0.993	13202.77
no_gamma_early_stop	424.108	1.24146	0.562775	0.994	20611.65
cand_always_enqueue	423.097	1.07849	0.562775	0.993	13201.37

2.4 工程上的优化

在该组消融实验中，我们进一步分析了 **距离计算实现方式、构建并行性以及 CPU 预取优化** 对整体性能的影响。Baseline 在当前配置下表现较为均衡，构建时间为 424.1 s，平均查询时间为 0.7588 ms，Recall@10 为 0.993，平均距离计算次数为 13202.8。

当采用 **float 距离计算（float_distance）** 替代量化距离时，构建时间上升至 538.8 s，平均查询时间显著增加至 1.0591 ms，增幅约 40%，同时中位数延迟也明显升高。尽管 Recall@10 略有提升（0.995），但平均距离计算次数几乎不变，说明性能下降主要来自单次距离计算代价的增加，而非搜索路径或访问规模的变化。这表明量化距离在保持排序一致性的前提下，对提升查询吞吐具有显著作用。

在 **串行构建（serial_build）** 的情况下，构建时间大幅上升至 1673.7 s，约为 Baseline 的 4 倍，而查询阶段的平均延迟、Recall 及距离计算次数几乎保持一致。这说明并行构建主要影响离线构建效率，对最终图结构质量及查询行为影响较小，但在大规模数据集上对工程可用性至关重要。

当 **关闭 CPU 预取（no_prefetch）** 时，构建时间略有下降，但平均查询时间上升至 0.9307 ms，中位数延迟也明显增加，而距离计算次数基本不变。这表明预取优化主要通过改善内存访问局部性来降低实

际查询延迟，而非减少搜索扩展规模。

综合来看，量化距离、并行构建与预取优化分别从计算吞吐、构建效率与内存访问三个层面提升了系统整体性能，均为当前版本中不可或缺的工程优化组成部分。

组	buildtime	avg_ms	med_ms	recall	avgcmp
baseline	424.1	0.758759	0.404159	0.993	13202.77
float_distance	538.806	1.05907	0.57018	0.995	13197.39
serial_build	1673.7	0.765464	0.402186	0.993	13201.29
no_prefetch	410.667	0.930724	0.498464	0.993	13195.34

2.5 参数设置

在参数选择阶段，为避免单一配置带来的偶然性影响，本文采用网格化遍历（grid search）的方式对关键参数进行系统调优。具体而言，我围绕底层图结构与查询策略中的核心参数进行组合测试：照顾到空间局部性，第 0 层最大度数参数 M_0取值为 {50, 52, 54, 56, 58, 60}，之后为保证 recall，构建阶段搜索宽度 $efConstruction$取值为 {300, 400, 500}，查询阶段早停控制参数 γ取值为 {0.17, 0.18, 0.19, 0.20, 0.21, 0.22}，我对上述参数空间中的所有组合进行了完整实验评估。

在每组参数配置下，我均记录构建时间、平均查询延迟、Recall@10 以及平均距离计算次数等指标，并重点关注在 Recall 基本稳定的前提下查询效率与距离计算规模的变化趋势。实验结果表明，随着 M_0 与 $efConstruction$的增大，图结构的可导航性整体提升，但构建代价随之上升；而 γ的取值则直接影响查询阶段早停的激进程度，在速度与准确率之间形成明显权衡。

综合多组实验结果，参数配置 $M_0 = 54$、$efConstruction = 500$、$\gamma = 0.21$ 在查询时间、距离计算次数与 Recall 之间取得了较为均衡的表现，既避免了过度扩展带来的冗余计算，又保持了稳定的近邻搜索精度。因此，本文后续实验均采用该参数组合作为默认配置。

M0	efC	gamma	build_sec	g_search	an_searc	recall10	t_comp_per_query
52	300	0.16	243.845	0.2843	0.2019	0.973	5009.32
52	300	0.17	242.249	0.3246	0.221	0.977	5918.36
52	300	0.18	241.93	0.417	0.2679	0.984	7097.28
52	300	0.19	239.665	0.5278	0.3227	0.986	8377.74
52	300	0.2	337.322	0.5846	0.3442	0.988	9163.56
52	300	0.21	242.729	0.759	0.4173	0.988	11877.5
52	300	0.22	243.079	0.9051	0.468	0.992	13939.83
52	400	0.16	328.208	0.2962	0.2076	0.979	5159.94
52	400	0.17	341.697	0.4383	0.256	0.98	6169.26
52	400	0.18	328.288	0.4073	0.2531	0.982	7345.58
52	400	0.19	329.376	0.5098	0.3022	0.985	8751.99
52	400	0.2	328.003	0.5674	0.3372	0.991	10251.85
52	400	0.21	330.26	0.744	0.4036	0.991	12197.81
52	400	0.22	329.865	0.8921	0.4769	0.993	14568.5
52	500	0.16	405.348	0.3155	0.2148	0.982	5339.88
52	500	0.17	412.593	0.3659	0.2443	0.982	6323.99
52	500	0.18	420.824	0.4369	0.268	0.985	7525.88
52	500	0.19	408.824	0.5339	0.3198	0.987	9025.77
52	500	0.2	406.496	0.6221	0.3568	0.989	10611.5
52	500	0.21	404.658	0.7541	0.4025	0.992	12666.29
52	500	0.22	421.205	0.9156	0.4797	0.995	15082.72
54	300	0.16	245.023	0.287	0.2009	0.972	5135.99
54	300	0.17	246.642	0.3777	0.2539	0.975	6123.29
54	300	0.18	246.482	0.4283	0.2644	0.98	7329.76
54	300	0.19	244.212	0.4857	0.297	0.984	8645.22
54	300	0.2	247.667	0.5715	0.3379	0.987	10242.78
54	300	0.21	249.824	0.7417	0.406	0.99	12032.47
54	300	0.22	247.78	0.9475	0.4891	0.99	14500.55
54	400	0.16	1235.4	0.3155	0.2202	0.979	5327.12
54	400	0.17	338.383	0.3608	0.2442	0.982	6359.68
54	400	0.18	335.747	0.4184	0.2575	0.985	7668.4
54	400	0.19	592.984	0.4943	0.285	0.988	9153.52
54	400	0.2	337.335	0.4986	0.2988	0.988	9159.48
54	400	0.21	336.204	0.7163	0.3785	0.992	12764.76
54	400	0.22	338.382	0.9205	0.4619	0.994	15242.05
54	500	0.16	420.715	0.3213	0.2201	0.98	5591.42
54	500	0.17	462.596	0.3763	0.2505	0.982	6509.64
54	500	0.18	441.227	0.4503	0.2886	0.987	7733.26
54	500	0.19	424.433	0.549	0.3271	0.987	9330.49
54	500	0.2	420.956	0.6169	0.3478	0.99	11078.68
54	500	0.21	1300.61	0.7443	0.3939	0.993	13199.4
54	500	0.22	424.124	0.9298	0.4688	0.995	15721.29
56	300	0.16	251.884	0.2902	0.201	0.975	5396.1
56	300	0.17	252.128	0.3576	0.2417	0.98	6388.11
56	300	0.18	249.564	0.4215	0.2682	0.983	7531.66
56	300	0.19	251.399	0.5088	0.2999	0.985	9004.26
56	300	0.2	250.337	0.5954	0.3556	0.988	10609.8
56	300	0.21	251.576	0.7171	0.3951	0.989	12725.43
56	300	0.22	249.856	0.8591	0.4463	0.994	14945.25
56	400	0.16	347.756	0.3076	0.2154	0.978	5528.61
56	400	0.17	344.277	0.3565	0.2386	0.981	6617.68
56	400	0.18	344	0.4679	0.2867	0.984	7938.47
56	400	0.2	343.964	0.6573	0.3819	0.989	11224.93
56	400	0.21	347.471	0.7492	0.3895	0.99	13409.29
56	400	0.22	346.519	0.9011	0.4652	0.992	15864.89
56	500	0.16	430.526	0.3123	0.2141	0.981	5808.59
56	500	0.17	435.603	0.3765	0.2435	0.982	6847.01
56	500	0.18	724.659	0.4524	0.2868	0.988	8102.71
56	500	0.19	436.006	0.536	0.3022	0.989	9711.52
56	500	0.2	338.832	0.5165	0.3088	0.988	9148.11
56	500	0.21	432.165	1.1329	0.6244	0.994	13679.69
56	500	0.22	438.289	0.9349	0.4731	0.996	16317.87
58	300	0.16	252.874	0.3017	0.2128	0.976	5464.86
58	300	0.17	255.509	0.3465	0.2327	0.98	6439.55
58	300	0.18	253.544	0.4251	0.2684	0.984	7725.82
58	300	0.19	252.877	0.5228	0.3047	0.986	9260.93
58	300	0.2	263.447	0.6359	0.3716	0.989	10943.66
58	300	0.21	257.255	0.7193	0.392	0.989	13099.75
58	300	0.22	257.032	0.8785	0.4537	0.993	15447.94
58	400	0.16	355.029	0.3119	0.2264	0.98	5765.22
58	400	0.17	353.586	0.3713	0.2463	0.982	6872.47
58	400	0.18	353.49	0.451	0.2758	0.985	8254.22
58	400	0.19	357.41	0.5574	0.3223	0.987	9953.34
58	400	0.2	352.742	0.6424	0.3634	0.989	11636.09
58	400	0.21	355.975	0.7955	0.4172	0.991	13861.26
58	400	0.22	360.143	0.9274	0.4754	0.992	16403.25
58	500	0.16	457.039	0.3159	0.2247	0.985	5798.24
58	500	0.17	445.799	0.3878	0.2514	0.984	6971.66
58	500	0.18	456.171	0.4547	0.2815	0.987	8335
58	500	0.19	449.7	0.5403	0.3115	0.988	10034.66
58	500	0.2	445.066	0.6571	0.3743	0.991	11899.55
58	500	0.21	449.284	0.7846	0.4105	0.993	14191.64
58	500	0.22	452.159	0.9437	0.4826	0.997	16860.65
60	300	0.16	261.755	0.3065	0.2162	0.976	5631.5
60	300	0.17	260.184	0.3604	0.2375	0.981	6646.82
60	300	0.18	259.393	0.4425	0.2789	0.984	7936.12
60	300	0.19	259.16	0.5222	0.311	0.986	9550.31
60	300	0.2	257.791	0.6277	0.3696	0.989	11279.25
60	300	0.21	258.029	0.7635	0.4247	0.991	13445.32
60	300	0.22	256.809	0.9154	0.4838	0.995	15814.64
60	400	0.16	357.086	0.3407	0.2341	0.98	6004.22
60	400	0.17	357.057	0.3867	0.2545	0.98	7150.68
60	400	0.18	361.284	0.462	0.275	0.984	8573.71
60	400	0.19	355.669	0.5718	0.3189	0.986	10307.25
60	400	0.2	358.743	0.6867	0.3883	0.988	12083.55
60	400	0.2	337.083	0.5631	0.3274	0.988	9159.91
60	400	0.21	357.142	0.8177	0.434	0.99	14382.08
60	400	0.22	356.714	0.9685	0.488	0.991	17005.21
60	500	0.16	451.862	0.3222	0.2179	0.985	6029.32
60	500	0.17	455.24	0.3871	0.2532	0.985	7213.15
60	500	0.18	452.357	0.4708	0.2927	0.988	8643.31

60	500	0.19	453.635	0.582	0.3353	0.989	10386.09
60	500	0.2	452.947	0.6885	0.3884	0.991	12336.92
60	500	0.21	454.487	0.8749	0.4609	0.993	14684.08
60	500	0.22	451.724	0.998	0.5038	0.997	17449.46

3 伪代码解析

- **Algorithm A：LID-based Two-pass Construction**

Input:

Base vectors X = {x_i}

EF_LID_BUILD, K_LID

Output:

Graph G

```
// Pass 1: Coarse graph for LID estimation
set efC ← EF_LID_BUILD
build graph G_coarse using default insertion order
for each node i do
    N_i ← first K_LID neighbors of i in G_coarse (level 0)
    D_i ← { dist(i, j) | j ∈ N_i }
    LID[i] ← EstimateLID(D_i)
end for

// Pass 2: Formal construction
order ← sort nodes by descending LID
reset graph
build final graph G using insertion order = order
```

- **Algorithm B：Reverse-edge Injection with Pruning**

Input:

Level-0 graph G0, max degree M0

Output:

Refined G0

```
for each node u in G0 do
    for each v in G0[u] do
        if u ∉ G0[v] then
            add u to G0[v]
            if |G0[v]| > M0 then
                G0[v] ← HeuristicPrune(v, G0[v], M0)
            end if
        end if
    end for
end for

for each node v do
    if |G0[v]| > M0 then
```

```
13:                 G0[v] ← HeuristicPrune(v, G0[v], M0)
14:         end if
15: end for
```

- **Algorithm C：Gamma-bounded Beam Search**

```
Input:
    query q, entry ep, ef, gamma
Output:
    Top-K neighbors

1: B ← max-heap (best results)
2: C ← min-heap (candidates)
3: push (dist(q, ep), ep) into B and C

4: while C not empty do
5:     (d_cur, u) ← pop C
6:     d_k ← max distance in B
7:     if d_cur > (1 + gamma) * d_k then
8:         break
9:     end if

10:    for each v in neighbors(u) do
11:        if v not visited then
12:            d ← dist(q, v)
13:            if |B| < ef or d < max(B) then
14:                insert (d, v) into B
15:                if |B| > ef then pop max(B)
16:            end if
17:            if d ⩽ (1 + gamma) * max(B) then
18:                push (d, v) into C
19:            end if
20:        end if
21:    end for
22: end while

23: return top K elements of B
```

- **Algorithm D：Heuristic Neighbor Selection**

```
Input:
    Candidate set C sorted by distance
    Degree limit M
Output:
    Selected neighbors S

1: S ← ∅
2: for each (d, v) in C do
```

```
3:      if |S| == M then break
4:      valid ← true
5:      for each u in S do
6:          if dist(u, v) < d then
7:              valid ← false; break
8:          end if
9:      end for
10:     if valid then add v to S
11: end for
12: return S
```

- **Algorithm E：Flattened Adjacency with Prefetch**

  For each node u:

  store neighbors as:

  [deg(u), v1, v2, ..., v_deg]

  During search:

  prefetch address of next neighbor vector before distance computation

- **Algorithm F：SQ16 L2 Distance**

  Input:

  quantized vectors a, b (int16)

  Output:

  squared L2 distance

```
1: sum ← 0
2: for i = 0 to dim step 16 do
3:      diff ← a[i:i+15] - b[i:i+15]
4:      sum ← sum + dot(diff, diff)
5: end for
6: return sum
```

说明：报告中所出现的数据均为本地所测得，并且用的是助教修改前的数据集（glove）。

目前提交的版本对应在测评机上 recall 为 98.91%，search 为 1.0577ms。

用更新前的数据集在本地测得的结果，对应到测评机上，recall 一般会降低 0.3~0.6%（此版本在本地 99.6%）不等，search_time 会变成原来的 1.5 倍(此版本在本地 0.718538ms)。此版本在本地的 avg_cmp 为 12970。

我在目前提交的版本上用更新后的数据集进行了补测（代码完全相同），在本地测得 avg_cmp 为 14362.490，较更新前的数据集对应的 avg_cmp 增加了 1500 次左右，recall 不变（99.6%），search_time 为 0.800284ms,特此进行说明。